Union-Find Data Structure

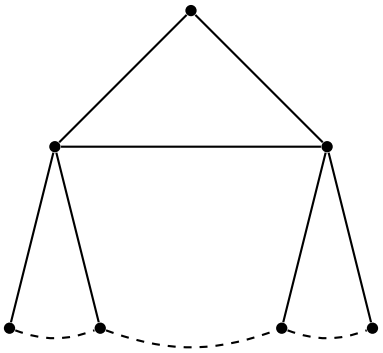
Lecture Notes Transcription

Introduction

In the last class, we talked about Kruskal's algorithm for computing minimum spanning trees. In the algorithm, there was an important step which actually dictates the running time of the algorithm, and that step was how to detect if there is a cycle that gets formed when we add a new edge in the current set of edges which were already selected as part of the min spanning tree.

If you recall, we would try to maintain the collection of components. So we would like to look at a data structure called the **Union-Find** data structure.

Let us look at Kruskal's algorithm at some point in time. It has picked a set of edges already which has formed some sort of forest as follows:



There should not be any cycle existing. Now, when the algorithm is trying to add a new edge, it needs to check if it forms a cycle or not. Suppose there is a new edge being added. If this edge forms a cycle, one way of detecting whenever a certain edge would form a cycle is to check if its two endpoints lie in the same tree of this forest. A tree is the same as the connected component, and so it suffices to check whether the two endpoints of an edge lie in our same connected component.

So, we have to somehow maintain the collection of connected components, so as the algorithm proceeds, the number of connected components reduces by one with every step. Initially, we started off with n connected components (each vertex being a component) and eventually, we will have only one connected component. This is how the algorithm proceeds.

Operations: Union and Find

We are now going to abstract this problem out and capture it as maintaining a disjoint collection of sets. So what's the setting now? I have a universe of elements, let us say $U = \{e_1, e_2, \ldots, e_n\}$ (n

elements in the universe). Initially, each element $e_i \in U$ is a set in itself, like $\{e_1\}, \{e_2\}, \ldots, \{e_n\}$. These elements in Kruskal's algorithm correspond to the vertices of the graph.

So what we are going to do now is maintain a collection of disjoint sets under the operation of:

1. `Union(S1, S2)`
2. `Find(x)`

What does `Union` do? Suppose we have two connected components and the edge that we add forms a link between these two connected components. Then the resulting thing would be one single connected component. So if the 1st component has $\{e_1, e_2, e_4\}$ in it and the 2nd component has $\{e_3, e_5, e_6\}$, then after union, we should not have these 2 sets. They should get replaced by 1 set which is $\{e_1, e_2, e_3, e_4, e_5, e_6\}$. So `Union` takes two sets as parameters: `Union(S1, S2)`.

The other operation `Find(x)` takes an element x as input and tells which set it belongs to. There is a unique set in which x lies because a collection of disjoint sets is always a partition of the universe.

So how do we use the definition of `Find(x)` to check whether the addition of an edge formed a cycle or not? Well, suppose the endpoints of the edge are x and y, we do `Find(x)` and `Find(y)`. If they both return the same set, then we conclude that it is forming a cycle; otherwise, if the sets are different, then it is not forming a cycle and the edge is added using the `Union(Find(x), Find(y))` operation.

Kruskal's Algorithm using Union-Find

If we write down Kruskal's algorithm now, it would look like this, assuming these two operations are available:

Algorithm 1 Kruskal's Algorithm

```
1: T = ∅
2: Sort the edges in increasing order of length. Let e_1, e_2, ..., e_m be the ordering.
3: for i = 1 to m do
4:     Let e_i = (u, v)
5:     if Find(u) ≠ Find(v) then
6:         T = T ∪ {e_i}
7:         Union(Find(u), Find(v))
8:     end if
9: end for
10: return T
```

We now need to understand what kind of data structure we need to use for maintaining the collection of disjoint sets so that these two operations, viz. `Union` and `Find`, can be done very quickly.

Before we proceed further, let's try to count how many times we need the `Union` operation and how many times we need the `Find` operation.

- **Number of `Union` operations:** Every time we add an edge, we do a union. How many edges can be there in the tree? Exactly $n - 1$. So we will have exactly $n - 1$ unions.

- **Number of `Find` operations:** For every edge we consider, we have to do two `Find()` operations. So in the worst case, how many edges do we need to consider? All the edges. So the number of `Find()` operations that we need to use is $2m$. (The loop in the algorithm runs from 1 to m, but you know you can always break out of the loop the moment $n-1$ edges are included).

Suppose `Union` takes U time and `Find` takes F time, then the total running time of Kruskal's algorithm is:

$$T = O(m \log m + U \cdot n + m \cdot F)$$

So we need to find a good data structure; by good, we mean one which will have small U and F.

Naive Approach: Linked Lists

What are the possible data structures we can use to maintain the collection of disjoint sets?

Linked list? Initially, we have as many lists as there are elements. Let us see how much time both `Union()` and `Find()` take if we use a linked list.

- **`Union`:** Union takes constant time $O(1)$. Even if the linked lists become large, we can keep track of the front and rear ends of each list (head and tail pointers), so we can combine two linked lists in constant time. So union will not take too much time.
- **`Find`:** Find takes O(size of the linked list), which could be $O(n)$ in the worst case.

If we do this, the running time of Kruskal's algorithm becomes:

$$T = O(m \log m + n + m \cdot n) \approx O(mn)$$

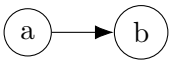
This is too large. We are looking for a data structure that should give running time something like $n \log n$ for these two operations. If we are looking for something like $n \log n$, the quantities U and F should not be more than $\log m$. So a linked list is not a good data structure.

Tree Representation

Which other data structure can we think of? A Tree? A heap? We will have a new data structure. I will just draw this new data structure and what it will look like, and then you will understand what is happening.

Suppose my universe is $U = \{a, b, c, d, e, f\}$ (six elements). Initially, the sets in my collection are all singleton sets. We have one node for each of these six sets.

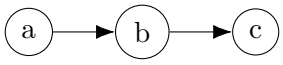
Now suppose we do `Union(Find(a), Find(b))`. Each node has a data field and a reference field. So what will happen is we will make one of these guys point to the other like this:



So when we say `Find(a)`, it will start from a and keep going up till it reaches the root; in this case, b is the root. How do we know it is the root? Well, its pointer points to itself (or is null). We are actually having a parent pointer with each node. So for each set, it is represented by one of the elements in the set. In some sense, all the elements of a set elect a leader. For example, in the set $\{a, b\}$, b is the leader.

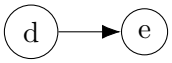
So when we say `Find(a)`, it returns to me a reference to the leader b.

What `Union` does is it takes the roots of the two trees and makes one point to the other. If we do `Union(Find(a), Find(c))`, we might decide to make b point to c.

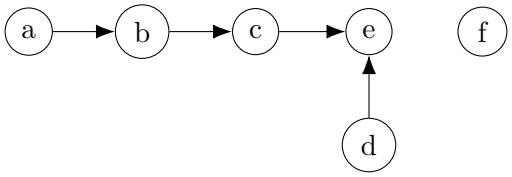


Now when I say `Find(a)` or even `Find(c)`, it returns c. `Find(b)` also returns c. It is best to return a reference because that can be used by the `Union` operation to merge the trees.

Suppose now we do `Union(Find(d), Find(e))`. What will we do? We will link up the roots for d and e. Let's say we decide to make d point to e.



Now if we do an operation like `Union(Find(a), Find(d))`, then what does `Find(a)` return? It returns a pointer to c. `Find(d)` returns a pointer to e. We need to link up these two roots. We can make c point to e or we can make e point to c, whichever we please. Let's say we decide to make c point to e. Then what we get is the following:



And of course, f would be sitting on its own.

I hope you understand what `Find` does. It starts from the element and keeps tracing up the pointers until it hits the root.

Now the question arises, how do we go to the element when I say `Find(a)`? Well, recall that we have a structure for the graph, maybe an adjacency matrix or adjacency list. Suppose we have an adjacency list representation. For each vertex in the adjacency list, we have another reference to the tree node above, so that we can access that particular node in the tree straight away.

What is the problem with this implementation? Let us see how much time both `Union` and `Find` operations take.

- **`Union:`** Union takes as input references to two root nodes and all it has to do is to modify one pointer to point to the other. So union takes $O(1)$ time.
- **`Find:`** How much time does find take? It can take a lot of time, because it might have to travel a long way to reach the root. In the worst case, it might take $O(n)$ time. For example, if we decide to merge first a and b, then b and c, then c and d, then d and e, and then e and f. In this order, things would go really bad (forming a linear chain).

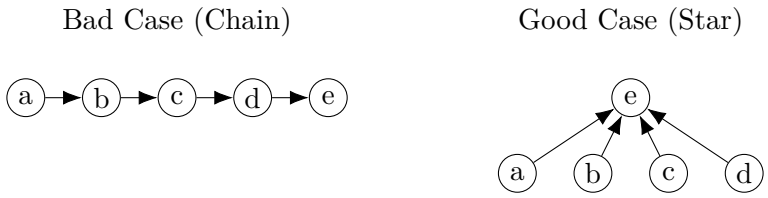
Optimized Union Strategies

So we have to do the union in a much cleaner manner. Recall that when we were linking the two roots, we had the option of linking the first one to the second, or the second to the first. We will now exploit that.

Union by Rank

We are going to use a rule called **Union by Rank**. This rule says that suppose we have two trees with n_1 and n_2 nodes in them. Then we make the lighter tree point to the heavier tree. How do we qualify a tree to be heavier? Well, by their number of nodes.

Suppose without loss of generality that $n_1 \leq n_2$. We will make the tree having n_1 nodes point to the root of the tree having n_2 nodes. Now we will not have this kind of scenario in which, for example, we have n elements, all of them forming a long chain. Rather it will be looking like the following:



What will be the height of this star tree? 1 only, and so `Find` will take very little time.

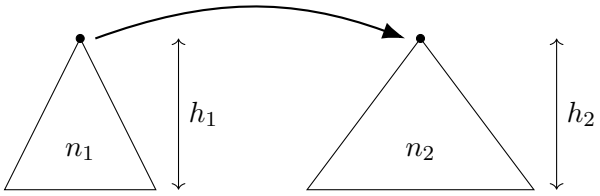
So we now need to see, if we use this rule, what could be the height of the tree in the worst case. How high can the tree become? I will argue that this rule of union by rank will lead to trees of height at most $\log n$. So the claim that we are going to make is this:

Claim: A tree with n nodes has height $\leq \log n$.

How do we prove this claim? Let me use induction to prove this. So at any stage, we are combining two trees, one with n_1 nodes and the other with n_2 nodes. Let W.L.O.G. $n_1 \leq n_2$.

Assume the induction hypothesis is true till this stage and:

$$h_1 \leq \log n_1$$
$$h_2 \leq \log n_2$$



Now we have to show that as a consequence of this, we will get a new tree with $(n_1 + n_2)$ nodes and whose height is $\leq \log(n_1 + n_2)$. Let's see if that is true.

What will be the height of the new tree? The height of the resulting tree would either be the height of the tree with n_2 nodes, or the height of the tree with n_1 nodes $+1$. i.e. height of the new tree $h = \max(h_2, h_1 + 1)$.

If this equals h_2, then:

$$h = h_2 \leq \log n_2 \leq \log(n_1 + n_2)$$

If this equals $h_1 + 1$, then:

$$h = h_1 + 1 \leq \log n_1 + 1 = \log(2n_1) \leq \log(n_1 + n_2)$$

because $n_1 \leq n_2 \implies 2n_1 \leq n_1 + n_2$.

Now, is the base case true? For $n = 1$, the height becomes 0. Let us define the height: the tree with only one node is of height 0. For $n = 2$, the height becomes 1. So the base case is true. We are counting the edges on the longest path from the leaves to the root.

It is to be noted that induction is on the number of nodes in the tree. So we are assuming the induction hypothesis to be true for trees having a certain number of nodes, and when we merge trees having these numbers of nodes, we get a tree with a larger number of nodes, and the statement is going to hold continuing to the new tree.

So this is all about Union by Rank. Rank actually means the number of nodes in the tree.

Union by Height

We can also do **Union by Height**. We can make the shallower tree point to the tree with the larger height. This approach will also work. Let's see what we are doing here.

We have two trees with heights h_1 and h_2 respectively, with $h_1 < h_2$. What we do in this case is we make the root of the tree with height h_1 point to the root of the tree with height h_2.

The claim that we are going to make now is this:

Claim: A tree of height h has at least 2^h nodes.

This is almost the converse of the earlier claim we were trying to prove. There we proved a tree with n nodes is of height $\leq \log n$, and here we are trying to prove if the height is h, then it has at least 2^h nodes. This will ultimately mean the tree can never be of height more than $\log n$, because if the height of the tree is more than $\log n$, it will have more than n nodes, which is not possible as there are only n nodes in the graph to begin with. So that will place a $\log n$ bound on the height of the tree.

Once again, by induction, we will prove the claim. Suppose the claim is true for all trees of height up to a certain number, and when we merge two trees, the resulting tree has height $h = \max(h_1, h_2)$ if $h_1 \neq h_2$, or $h_1 + 1$ if $h_1 = h_2$. Let h be the height of the new tree.

The number of nodes in the new tree is:

$$\text{Nodes} = n_1 + n_2 \geq 2^{h_1} + 2^{h_2}$$

If $h_1 < h_2$, then $h = h_2$.

$$n_1 + n_2 \geq 2^{h_2} = 2^h$$

If $h_1 = h_2$, then $h = h_1 + 1$.

$$n_1 + n_2 \geq 2^{h_1} + 2^{h_1} = 2 \cdot 2^{h_1} = 2^{h_1+1} = 2^h$$

So the number of nodes $\geq 2^h$. The proofs are very similar; we are just turning them around. So both of these schemes can be used to do the unions.

Time Complexity Analysis

Given this, how much time does `Union` take?

`Union`: Constant time $O(1)$. We just need to update the reference. To achieve this in union by rank, we need to keep track of the number of nodes or height of the tree in the root node, and this information is very easy to maintain. If we do union by rank, we just add the number of nodes in both the trees and keep it in the root node of the larger tree. On the other hand, if we do union by height, we need to update the height at the root of the taller tree to $\max(h_1, h_2 + 1)$.

Now how much time does `Find` take?

`Find`: Takes no more than $O(\log n)$ time.

So the total time taken by Kruskal's algorithm thus becomes:

$$T = O(m \log m + n + m \log n)$$

What is the maximum value of m? Well, in a simple connected graph, maximum value of $m \leq \frac{n(n-1)}{2} < n^2$. So, $\log m \leq 2 \log n$. Thus $\log m = O(\log n)$.

So,

$$T = O(m \log m) \approx O(m \log n)$$